

Metric3D v2

A Versatile Monocular Geometric Foundation Model
for Zero-Shot Metric Depth and Surface Normal Estimation

Presented by J.C. Vaught

CSCE 763 — Digital Image Processing
University of South Carolina

Spring 2026

Mu Hu, Wei Yin, Chi Zhang, Zhipeng Cai, Xiao Long, Hao Chen,
Kaixuan Wang, Gang Yu, Chunhua Shen, Shaojie Shen

IEEE TPAMI, Vol. 46, No. 12, 2024

What Can You Measure from a Single Photo?

Goal: From one RGB image, recover per-pixel

- ▶ **Metric depth** — distance in meters from camera to each point
- ▶ **Surface normals** — 3D orientation of each surface patch

Demo output:
RGB | Depth | Normals
(replace with inference result)

Connection to course: The pinhole camera model (Ch. 2) relates 3D world coordinates to 2D image coordinates via focal length f :

$$u = f \frac{X}{Z}, \quad v = f \frac{Y}{Z}$$

Without knowing f , depth is ambiguous.

Why Is This Hard? — Metric Ambiguity

Key insight: Two cameras with different focal lengths can produce *identical* images of a scene at different depths.

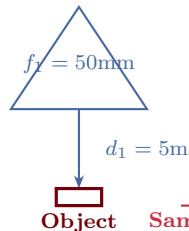
The apparent depth scales with focal length:

$$d_a = \hat{S} \cdot \frac{\hat{f}}{\hat{S}'}$$

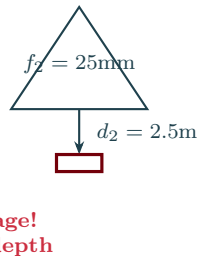
where $\hat{S}$ is real-world size, $\hat{f}$ is focal length, and $\hat{S}'$ is projected size.

A network trained on mixed datasets with different cameras sees **conflicting depth labels** for visually identical patches.

Camera 1: f_1



Camera 2: f_2

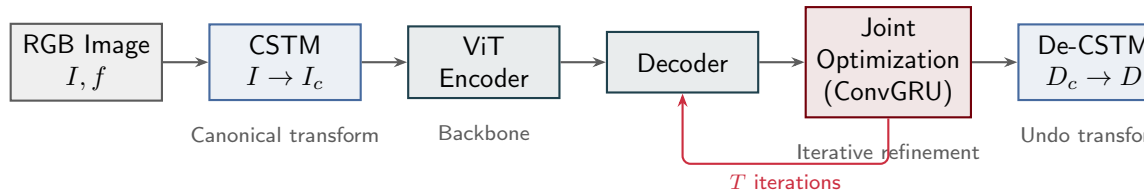


Depth Estimation Taxonomy



Metric3D v2 achieves *zero-shot metric depth* by explicitly modeling camera intrinsics, unlike methods that learn only relative ordering.

Architecture Overview



- **Modular:** Any backbone (ViT-S, ViT-L, ViT-G) plugs in; CSTM is backbone-agnostic
- **Two key innovations:** Canonical Camera Space Transform (CSTM) and Joint Depth-Normal Optimization

Focal Length Is Everything

The paper establishes two critical observations:

O1: Sensor/pixel size does not matter.

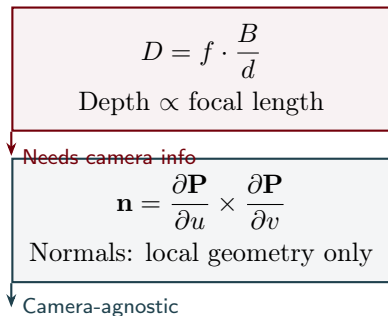
Only the ratio of focal length to pixel size matters, and this is captured by the focal length in pixels f_{px} .

O2: Focal length is vital for metric depth.

Different focal lengths cause different depth scales. The network must compensate, or depth predictions collapse across diverse cameras.

Normals are metric-agnostic:

Surface normals depend only on local geometry, not on absolute depth scale. This



Canonical Camera Space Transform — Label Space

Idea: Map all training data to a “canonical” camera with fixed focal length $f^c = 1000$ px.

CSTM_{label} transforms depth annotations:

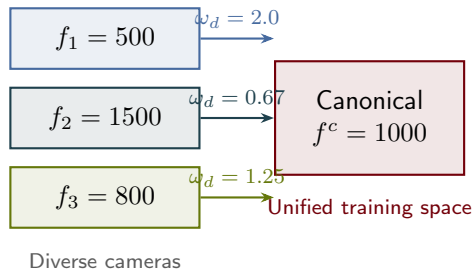
$$D_c^* = \omega_d \cdot D^*, \quad \omega_d = \frac{f^c}{f}$$

If $f < f^c$: depth is *scaled up* (wide-angle cameras see closer).

If $f > f^c$: depth is *scaled down*.

At inference, **De-CSTM** reverses:

$$D = \frac{D_c}{\omega_d} = D_c \cdot \frac{f}{f^c}$$



Canonical Camera Space Transform — Image Space

CSTM_{image} resizes the input image instead of scaling labels:

$$I_c = \mathcal{T}(I, \omega_r), \quad \omega_r = \frac{f^c}{f}$$

Why resize? After resizing, objects in I_c appear at the angular size they would subtend through the canonical camera. The encoder sees a consistent “canonical viewpoint.”

Both paths are equivalent:

- ▶ CSTM_{label}: scale depth labels, keep images
- ▶ CSTM_{image}: resize images, keep depth labels

In practice, the paper uses **CSTM**.

Wide-angle: $f = 500$



$\omega_r = 2$
→

Canonical: $f^c =$



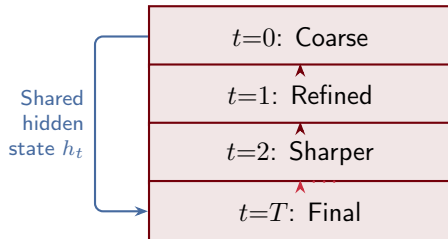
Joint Depth-Normal Optimization

Novel contribution: First method to jointly optimize metric depth and surface normals through an iterative process.

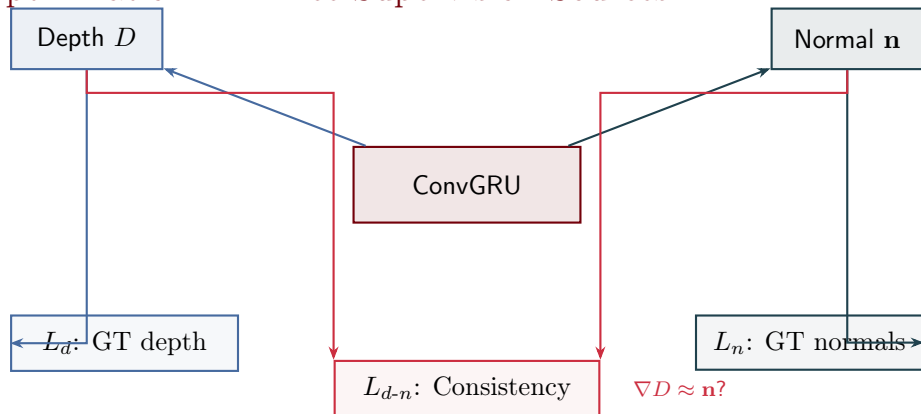
ConvGRU iterative refinement:

1. Decoder produces initial coarse estimates $D_0, \mathbf{n}_0$
2. At each iteration t :
 - ▶ Compute update Δ_t from hidden state
 - ▶ $D_{t+1} = D_t + \Delta_{D,t}$
 - ▶ $\mathbf{n}_{t+1} = \mathbf{n}_t + \Delta_{\mathbf{n},t}$
3. After T steps: final refined output

Why jointly? Depth and normals are geometrically coupled. Sharing a hidden state lets each task inform the other.



Joint Optimization — Three Supervision Sources



Converts depth gradients to pseudo-normals; penalizes disagreement

Key insight: Only $\sim 20\text{K}$ outdoor images have GT normals. The consistency loss L_{d-n} lets the model learn normals from depth-only datasets by enforcing geometric agreement.

Random Proposal Normalization Loss (RPNL)

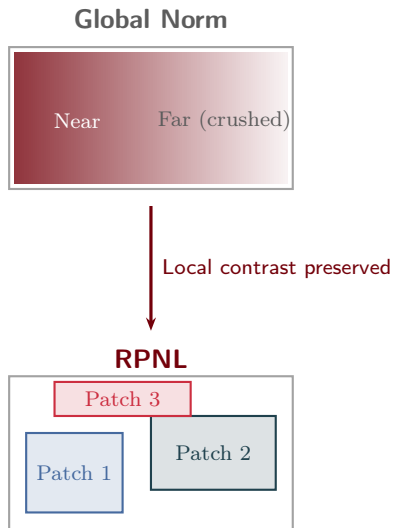
Problem: Standard depth losses normalize globally. Close objects dominate; distant detail is lost.

Analogy from course: This is the same problem *adaptive histogram equalization* solves for image contrast. Global normalization washes out local structure.

RPNL solution:

1. Randomly crop K patches from the depth map
2. Normalize depth within each patch independently
3. Compute loss per patch, then average

This forces the network to preserve *local*



Loss Function Summary

Total loss:

$$L = \underbrace{0.5 L_d}_{\text{depth}} + \underbrace{1.0 L_n}_{\text{normals}} + \underbrace{0.01 L_{d-n}}_{\text{consistency}}$$

$$L_d = L_{\text{PWN}} + L_{\text{VNL}} + L_{\text{silog}} + L_{\text{RPNL}}$$

Pair-wise normal | Virtual normal | Scale-invariant log | Random proposal

$$L_n = 1 - \langle \hat{\mathbf{n}}, \mathbf{n}^* \rangle \quad (\text{cosine similarity})$$

L_{d-n} : penalizes disagreement between ∇D (pseudo-normals from depth) and predicted $\mathbf{n}$

Note: L_{d-n} weight is small (0.01) because pseudo-normals from predicted depth

Training at Scale

16M
images

16
datasets

10K+
camera models

48 × A100
GPUs

Indoor: NYUv2, ScanNet, Taskonomy, ...
Outdoor: KITTI, DDAD, Waymo, Lyft, ...
Mixed: DIML, 3D Ken Burns, BlendedMVS, ...

Critical data gap: Only $\sim 20\text{K}$ outdoor images have GT surface normals (mostly from ScanNet indoor). This motivates the depth-normal consistency loss L_{d-n} as a way to learn normals from depth-only outdoor data.

Benchmark Results

Zero-shot metric depth (no fine-tuning on target dataset):

Dataset	AbsRel ↓	δ_1 ↑
NYUv2	0.063	0.975
KITTI	0.043	0.989
ScanNet	0.069	0.970
DIODE Out	0.308	0.832

Key: Zero-shot results that *beat* methods fine-tuned on the target dataset. CSTM enables generalization across camera types.

Demo output:

Our inference results
on test images

RGB | Depth | Normals
(replace with inference result)

Downstream Applications

3D Reconstruction

Back-project depth to point cloud:

$$X = \frac{(u - u_0) \cdot D}{f}$$

$$Y = \frac{(v - v_0) \cdot D}{f}$$

$$Z = D$$

Table 6: competitive with MVS methods using a *single* image.

SLAM Improvement

Using Metric3D v2 as depth prior in visual SLAM:

- ▶ 10× reduction in drift (Table 7)
- ▶ Enables metric-scale monocular SLAM

Our Point Cloud Demo

*Point cloud
rendering*

(replace with
PLY render)

Strengths

1. **CSTM is elegant and modular.** A single scaling factor $\omega = f^c/f$ handles all camera diversity. No architecture changes needed; works with any backbone.
2. **Learning normals from depth-only data.** The consistency loss L_{d-n} sidesteps the scarcity of outdoor normal annotations ($\sim 20K$ images). The network learns normals as a *byproduct* of depth prediction.
3. **Comprehensive evaluation.** Tested on 16+ benchmarks spanning indoor, outdoor, in-the-wild, and downstream tasks (3D reconstruction, SLAM, metrology).
4. **Immediately usable.** Available via PyTorch Hub with multiple model sizes (ViT-S, ViT-L, ViT-G). One line of code to run inference.

Limitations and Open Questions

1. **Requires camera intrinsics at inference.** Unlike UniDepth (CVPR 2024) and Depth Pro (Apple, 2024), which estimate intrinsics internally. Users must provide or estimate f .
2. **Single-frame only.** No temporal consistency for video. Sequential frames can produce flickering depth maps.
3. **Outdoor normal data scarcity persists.** The consistency loss helps but does not fully close the gap; outdoor normal quality lags indoor.
4. **Canonical focal length $f^c = 1000$ is ad hoc.** The paper provides no theoretical justification for this choice. Sensitivity analysis would strengthen the claim.
5. **Newer competitors are closing the gap.**
 - ▶ **MoGe** (CVPR 2025): jointly predicts 3D points + normals + FOV
 - ▶ **Depth Pro**: metric depth without intrinsics at 0.3s per image

Key Takeaways

1. **CSTM** — canonical camera transform

2. **Joint optimization** — depth + normals

3. **RPNL** — local depth normalization

Big idea: Transforming the *data space* (CSTM) is simpler and more effective than encoding camera parameters into the network architecture.

Geometric foundation models trained on massive diverse data can rival — and often beat — task-specific models fine-tuned on individual benchmarks.

Questions?